

Software Requirements Specification

End-to-End Encrypted Messaging Architecture

Prepared for: Cross-Platform App Development

May 15, 2026

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Document Conventions	2
1.3	Intended Audience	2
1.4	Product Scope	2
1.5	Definitions, Acronyms, and Abbreviations	2
2	Overall Description	4
2.1	Product Perspective	4
2.2	Product Functions	4
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
3	System Features	6
3.1	Device-Bound Cryptographic Identity	6
3.2	Offline-First Messaging Engine	6
3.3	Encrypted Control Messages (Metadata Protection)	6
3.4	Encrypted Media Uploads	7
4	Non-Functional Requirements	8
4.1	Performance Requirements	8
4.2	Security Requirements	8
4.3	Reliability & Availability	8

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive architectural and functional overview of the End-to-End Encrypted (E2EE) Messaging Application. This document details the expected behaviors, interfaces, and constraints of the system, serving as the definitive blueprint for the development of the frontend monorepo (React Native), the backend orchestration layer (Spring Boot), and the cryptographic core (C++).

1.2 Document Conventions

This document adheres to the IEEE 830-1998 standard for SRS documentation. All cryptographic references assume the use of standard, peer-reviewed algorithms (Curve25519, AES-256-GCM). The term “Local-First” denotes a system architecture where the client’s local database (WatermelonDB) serves as the primary source of truth, with asynchronous server synchronization.

1.3 Intended Audience

This document is intended for:

- **Software Engineers:** Backend, frontend, and systems developers responsible for implementing the architecture.
- **Security Auditors:** Professionals reviewing the cryptographic boundaries and data flow.
- **Project Managers:** For tracking milestone deliverables.

1.4 Product Scope

The proposed system is a highly secure, cross-platform messaging application offering 1-on-1 chats, group chats, broadcast channels, and rich media sharing. The application enforces a strict Zero-Knowledge paradigm, ensuring that the orchestration server routing the messages cannot read the plaintext content, metadata, or media attachments. The system will deploy across Android, Web, and Desktop environments.

1.5 Definitions, Acronyms, and Abbreviations

E2EE	End-to-End Encryption
JWT	JSON Web Token
STOMP	Simple Text Oriented Messaging Protocol
JSI	JavaScript Interface (React Native)
Wasm	WebAssembly
ECDH	Elliptic Curve Diffie-Hellman

HKDF	HMAC-based Extract-and-Expand Key Derivation Function
HAL	Hardware Abstraction Layer

2 Overall Description

2.1 Product Perspective

The messaging application operates as a standalone ecosystem. It acts as a federated client-server model where the server is a stateless, untrusted relay, and the clients hold absolute cryptographic authority. It replaces conventional cloud-based messaging by ensuring privacy through cryptographic mathematics rather than policy.

2.2 Product Functions

The primary functions of the system include:

- **Account Management:** Cryptographic identity generation and public-key publication.
- **Direct Messaging:** Secure 1-on-1 text and media exchange.
- **Group Messaging:** Sender-key based multi-party encryption for scalable group chats.
- **Media Management:** Encrypted blob uploads and key sharing.
- **State Synchronization:** Cursor-based offline-first delta sync logic.

2.3 User Classes and Characteristics

- **Standard Users:** Individuals requiring secure communication without needing to understand the underlying cryptographic complexities.
- **Enterprise Users:** Organizations requiring auditability of their secure enclave usage.

2.4 Operating Environment

The software will operate within the following environments:

- **Client (Mobile):** Android 8.0+ (API Level 26 and above).
- **Client (Desktop/Web):** Modern Web Browsers (Chrome 90+, Firefox 88+), Windows 10/11, macOS 12+ via Electron.
- **Server:** Linux-based OS (Ubuntu 22.04 LTS), Java 21 Runtime Environment, PostgreSQL 16, Redis 7, MinIO.

2.5 Design and Implementation Constraints

- **Cryptographic Isolation:** JavaScript code MUST NOT have access to private keys or derived symmetric keys in raw memory. All operations must occur within the C++ memory boundary.
- **Storage Limitations:** Mobile devices have limited storage; therefore, the local database must efficiently prune or archive decrypted messages.

- **Network Latency:** The system must handle unreliable mobile networks gracefully through its Local-First design.

3 System Features

3.1 Device-Bound Cryptographic Identity

Description: To support true Zero-Knowledge multi-device usage, identities are bound strictly to devices, not user accounts. Upon registration or device provisioning, the application generates a unique public/private key pair (Curve25519) for that specific device.

Functional Requirements:

1. The client shall generate a 256-bit private key securely bound to the device’s hardware enclave.
2. The client shall transmit the public key to the server, appending it to the User’s list of authorized devices.
3. Senders must retrieve the public keys for *all* of a recipient’s active devices and encrypt the message pairwise for each device (the “Session” protocol approach).

3.2 Offline-First Messaging Engine

Description: Users must be able to read past messages and draft new messages regardless of network connectivity.

Functional Requirements:

1. The UI shall perform all reads and writes directly against the local WatermelonDB instance.
2. Messages drafted offline shall be marked as ‘PENDING’ and queued for transmission upon network restoration.
3. The system shall utilize STOMP WebSockets to perform a ‘Delta Sync’ upon reconnection, utilizing an auto-incrementing, monotonic BIGINT sequence ID (rather than timestamps) to prevent race conditions.
4. The system shall maintain a Local Sender Key Vault within the client’s local database to securely cache incoming group Sender Keys. This ensures that massive influxes of group messages can be decrypted instantaneously upon network reconnection without requiring sequential key-fetch requests to the orchestration server.

3.3 Encrypted Control Messages (Metadata Protection)

Description: To prevent metadata leakage regarding user activity, events like read-receipts and typing indicators must not be sent in plaintext.

Functional Requirements:

1. Read-receipts and typing states shall be encapsulated as E2EE payloads (Control Messages) rather than STOMP headers.
2. The orchestration server shall route these control messages blindly without knowing their semantic intent.

3.4 Encrypted Media Uploads

Description: Images and videos must be encrypted before transmission and stored securely on the server.

Functional Requirements:

1. The client shall request a pre-signed MinIO URL from the backend.
2. The C++ core shall symmetrically encrypt the media file using a randomly generated AES-256-GCM key.
3. The client shall upload the ciphertext blob directly to MinIO.
4. The symmetric decryption key for the media shall be appended to the recipient's secure message payload.

4 Non-Functional Requirements

4.1 Performance Requirements

- The Java Spring Boot backend shall utilize Virtual Threads to support up to 100,000 concurrent WebSocket connections per instance.
- Message delivery latency (Client A $\rightarrow$ Server $\rightarrow$ Client B) shall not exceed 200ms under standard network conditions.

4.2 Security Requirements

- **Zero-Knowledge Routing:** The database schema shall not contain any plain text message data. The `ciphertext` column in PostgreSQL must be rigorously maintained as raw bytes.
- **Perfect Forward Secrecy (PFS):** The key derivation process shall ensure that compromised session keys do not compromise past communications.

4.3 Reliability & Availability

- The messaging infrastructure shall be containerized via Docker and orchestrated via Docker Compose to ensure rapid deployment and high availability.
- The system shall guarantee at-least-once delivery; messages shall only be removed from the server's pending queue after an explicit client Acknowledgment (ACK).